

CS 449 Peg Solitaire BrainVita

Sprint 0 Report

Devan Sai D 16404909 | CS 449 | Submitted September 18, 2026

GitHub Repository: <https://github.com/devansai10/cs449-peg-solitaire>

For Sprint 0, I selected the tools for my Peg Solitaire BrainVita project and created two small programs: a player-rating program tested with JUnit and a Java Swing interface for selecting game settings. This work helped me practice unit testing and GUI programming before developing the game in later sprints.

1 Key Decisions of the Solitaire Project

Topic	Decision
Object-oriented programming language	Java 21. I chose Java because it supports object-oriented programming, which will help me organize the game logic and different board types.
GUI library	Java Swing. It is included with Java and provides the controls needed for a standalone desktop application.
IDE	Visual Studio Code. I use it to write and edit the code and run Maven tests in its terminal.
xUnit framework	JUnit Jupiter 5.10.3, with Maven Surefire 3.2.5 to run the tests.
Programming style guide	Google Java Style Guide. I will use it throughout the project.
Project hosting site	GitHub.com. My repository link is listed at the top of this report.
Build and dependency management	Maven. The pom.xml file stores the Java version, dependencies, and build settings.
Design approach	I will keep the game rules separate from the GUI so they can be tested and reused across board types.

Programming style practices

I choose the Google Java Style Guide to keep my code consistent and easy to read. My code uses two-space indentation, explicit imports, and braces around control statements. Class names use UpperCamelCase, while methods and variables use lowerCamelCase. I keep one top-level class in each file and limit lines to 100 characters. [1]

For Sprint 0, I focused on selecting tools, writing unit tests, and creating a basic GUI. In later sprints, I plan to add board initialization, manual play, autoplay, randomization, and record/replay, as described in the project overview.

2 Unit Testing

I selected the JUnit 5 User Guide, version 5.10.3, as the tutorial for this section. Its Writing Tests and Assertions sections explain the features used in my tests: `@Test` marks a test method, `assertEquals` compares the expected and actual results, and `assertThrows` checks whether an expected exception is thrown. [2]

My unit-testing example is `PlayerRating`, a small class that returns a rating based on the number of pegs remaining. I used the ratings described in the project overview. The class rejects zero and negative values because it expects at least one remaining peg. It only calculates the rating; it does not determine when the game ends.

Test input	Expected outcome
1 peg	Outstanding
2 pegs	Very Good
3 pegs	Good
4 and 32 pegs	Average
0 pegs	<code>IllegalArgumentException</code>
-1 pegs	<code>IllegalArgumentException</code>

I ran the six JUnit tests using Maven in the VS Code terminal. All six passed, with no failures, errors, or skipped tests. The screenshot below shows the results from September 17, 2026.

```
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ solitaire-sprint0 ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.2.5:test (default-test) @ solitaire-sprint0 ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running edu.cs449.solitaire.PlayerRatingTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.034 s -- in edu.cs449.solitaire.PlayerRatingTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.888 s
[INFO] Finished at: 2026-09-17T19:12:19-05:00
[INFO]
devansai > |
```

Figure 1. Six passing JUnit tests, with no failures or errors.

Unit Testing Source Code

PlayerRating.java

```
package edu.cs449.solitaire;

/** Converts the number of pegs remaining into a player rating. */
public final class PlayerRating {
    /** Returns a rating for a positive peg count; rejects zero and negative counts. */
    public String rate(int pegsRemaining) {
        if (pegsRemaining < 1) {
            throw new IllegalArgumentException("At least one peg must remain.");
        }
        return switch (pegsRemaining) {
            case 1 -> "Outstanding";
```

```

        case 2 -> "Very Good";
        case 3 -> "Good";
        default -> "Average";
    };
}
}

```

PlayerRatingTest.java

```
package edu.cs449.solitaire;
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
```

```
import org.junit.jupiter.api.Test;
```

```
class PlayerRatingTest {
    private final PlayerRating rating = new PlayerRating();
```

```

    @Test
    void onePegReceivesOutstanding() {
        assertEquals("Outstanding", rating.rate(1));
    }

```

```

    @Test
    void twoPegsReceiveVeryGood() {
        assertEquals("Very Good", rating.rate(2));
    }

```

```

    @Test
    void threePegsReceiveGood() {
        assertEquals("Good", rating.rate(3));
    }

```

```

    @Test
    void fourOrMorePegsReceiveAverage() {
        assertEquals("Average", rating.rate(4));
        assertEquals("Average", rating.rate(32));
    }

```

```

    @Test
    void zeroPegsAreRejected() {
        assertThrows(IllegalArgumentException.class, () -> rating.rate(0));
    }

```

```

    @Test
    void negativePegsAreRejected() {
        assertThrows(IllegalArgumentException.class, () -> rating.rate(-1));
    }

```

3 GUI Programming

For the GUI exercise, I chose a simple Solitaire settings window. It lets the user select a board type and a recording preference, then click Preview settings to display those choices. This is a Sprint 0 demonstration; the controls do not yet start a game or record moves.

Required element	Implementation
Text	JLabel title, instructions, board heading, and status message.
Lines	Two horizontal JSeparator components.
Check box	JCheckBox labeled Record game.

Radio buttons

English, Hexagon, and Diamond JRadioButton controls grouped in a ButtonGroup so only one is selected.

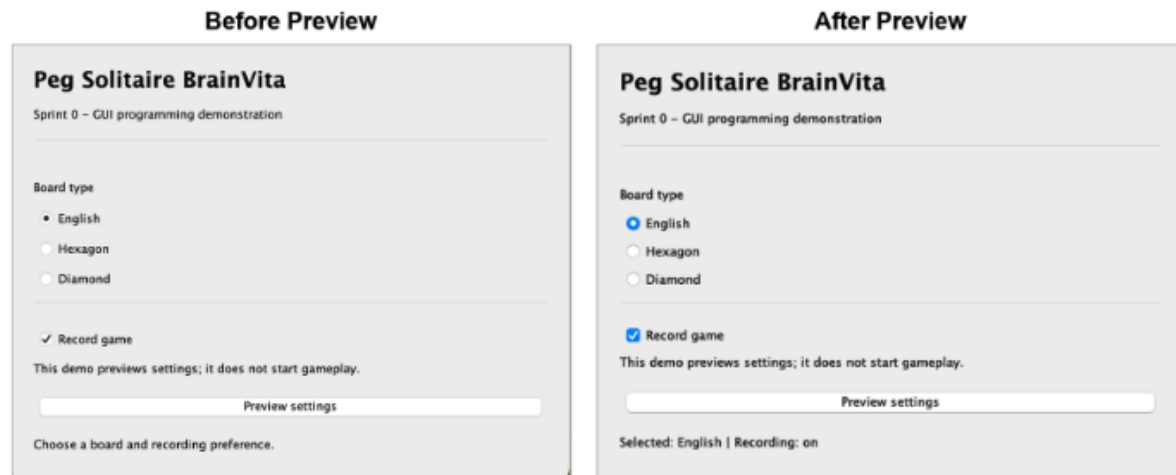


Figure 2. Swing settings window before and after clicking Preview settings; the updated status confirms English is selected and recording is on.

I used BorderLayout and GridLayout to organize the window. A ButtonGroup keeps the board choices mutually exclusive, while the check box provides an independent on/off setting. The Preview settings button calls `updateStatus()` to update the label. `SwingUtilities.invokeLater` creates the interface on Swing's event dispatch thread. [3, 4]

GUI Source Code

SolitaireSetupDemo.java, part 1 of 2

```
package edu.cs449.solitaire;

import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.GridLayout;
import javax.swing.BorderFactory;
import javax.swing.ButtonGroup;
import javax.swing.JButton;
import javax.swing.JCheckBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JSeparator;
import javax.swing.SwingUtilities;

/** Demonstrates Sprint 0 GUI components; full gameplay is added in later sprints. */
public final class SolitaireSetupDemo {
    private final JLabel status = new JLabel("Choose a board and recording preference.");
    private final JCheckBox recordGame = new JCheckBox("Record game", true);
    private final ButtonGroup boardTypes = new ButtonGroup();

    private void showWindow() {
        JFrame frame = new JFrame("CS 449 - Sprint 0 Peg Solitaire BrainVita");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel content = new JPanel(new BorderLayout(16, 16));
```

```

content.setBorder(BorderFactory.createEmptyBorder(24, 24, 24, 24));

JPanel heading = new JPanel(new GridLayout(0, 1, 0, 8));
JLabel title = new JLabel("Peg Solitaire BrainVita");
title.setFont(title.getFont().deriveFont(Font.BOLD, 24f));
heading.add(title);
heading.add(new JLabel("Sprint 0 - GUI programming demonstration"));
heading.add(new JSeparator());
content.add(heading, BorderLayout.NORTH);

JPanel options = new JPanel(new GridLayout(0, 1, 0, 10));
options.add(new JLabel("Board type"));
for (String name : new String[] { "English", "Hexagon", "Diamond" }) {
    JRadioButton button = new JRadioButton(name, name.equals("English"));
    button.setActionCommand(name);
    boardTypes.add(button);
    options.add(button);
}

```

GUI Source Code Continued

SolitaireSetupDemo.java, part 2 of 2

```

}
options.add(new JSeparator());
options.add(recordGame);
options.add(new JLabel("This demo previews settings; it does not start gameplay."));
content.add(options, BorderLayout.CENTER);

JPanel footer = new JPanel(new GridLayout(0, 1, 0, 12));
JButton preview = new JButton("Preview settings");
preview.addActionListener(event -> updateStatus());
footer.add(preview);
footer.add(status);
content.add(footer, BorderLayout.SOUTH);

frame.setContentPane(content);
frame.setMinimumSize(new Dimension(610, 480));
frame.pack();
frame.setLocationRelativeTo(null);
frame.setVisible(true);
}

private void updateStatus() {
    String board = boardTypes.getSelection().getActionCommand();
    String recording = recordGame.isSelected() ? "on" : "off";
    status.setText("Selected: " + board + " | Recording: " + recording);
}

/** Creates Swing components on the event dispatch thread. */
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new SolitaireSetupDemo().showWindow());
}
}

```

Build and run

To run the examples, open the project folder with JDK 21 and Maven available, then use these commands:

```

mvn test
java -cp target/classes edu.cs449.solitaire.SolitaireSetupDemo

```

References

- [1] Google Java Style Guide. <https://google.github.io/styleguide/javaguide.html>
- [2] JUnit 5 User Guide 5.10.3. <https://docs.junit.org/5.10.3/user-guide/index.html>

[3] Oracle Swing Tutorial: Buttons, Check Boxes, and Radio Buttons.

<https://docs.oracle.com/javase/tutorial/uiswing/components/button.html>

[4] Oracle Swing Tutorial: Separators.

<https://docs.oracle.com/javase/tutorial/uiswing/components/separator.html>